

تحلیل و طراحی الگوریتم‌ها

پاسخ‌نامه تمرین سری اول

نیم‌سال: زمستان ۱۴۰۵

نام استاد: دکتر ملکی مجد

تیم حل تمرین: امیرحسین رستگار، مهدی تقی دولابی، سارا احمدی

سؤال اول - تاس‌ها

فرض کنید n عدد تاس داریم که هر کدام دارای m وجه با مقادیر 1 تا m هستند. می‌خواهیم این تاس‌ها را پرتاب کنیم و مجموع اعداد به دست آمده از وجه‌های تاس‌ها برابر با مقدار مشخص x شود. الگوریتمی طراحی کنید که تعداد تمام حالت‌های ممکن برای رسیدن به مجموع x را محاسبه کند. منظور از حالت، یک ترتیب مشخص از مقادیر وجه‌های تاس‌ها است که مجموع آن‌ها برابر x شود. به عنوان مثال اگر دو تاس شش‌وجهی داشته باشیم و مقدار هدف $x = 5$ باشد، حالت‌های ممکن شامل $(1, 4)$، $(3, 2)$، $(4, 1)$ و $(2, 3)$ هستند؛ بنابراین تعداد حالت‌ها برابر ۴ است.

پاسخ تشریحی

برای حل این مسئله، تمام حالت‌های ممکن برای رسیدن به مجموع هدف x بررسی می‌شوند. در هر مرحله مقدار وجه یکی از تاس‌ها انتخاب شده و تعداد حالت‌های ممکن برای تاس‌های باقی‌مانده و مجموع باقی‌مانده به صورت بازگشتی محاسبه می‌شود. بنابراین در هر گام، تعداد تاس‌های باقی‌مانده یک واحد کاهش یافته و مقدار وجه انتخاب‌شده از مجموع هدف کم می‌شود.

حالت‌های پایه (Base Cases):

- اگر تمام تاس‌ها استفاده شده باشند و مجموع هدف نیز صفر باشد ($f(0, 0) = 1$).
- اگر تمام تاس‌ها استفاده شده باشند ولی به مجموع هدف نرسیده باشیم ($f(0, x) = 0$ به ازای $x \neq 0$).

- اگر مجموع هدف منفی شود ($f(n,x)=0$ به ازای $x<0$).

رابطه بازگشتی: برای هر مقدار وجه k از 1 تا m داریم:

$$f(n,x)=\sum_{k=1}^{m} f(n-1,x-k)$$

که در آن n تعداد تاس‌های باقی‌مانده و x مجموع باقی‌مانده است.
روش یادسپاری (Memoization): در روش بازگشتی ساده، ممکن است یک زیرمسئله چندین بار محاسبه شود. چون تنها دو پارامتر n و x تغییر می‌کنند، یک جدول دوبعدی با نام $\text{memo}[n][x]$ به ابعاد $(n+1)\times(x+1)$ ایجاد می‌کنیم. ابتدا تمام مقادیر جدول را برابر -1 قرار می‌دهیم تا مشخص شود کدام حالت‌ها هنوز محاسبه نشده‌اند.
روش جدول‌بندی (Tabulation): در این روش جدول DP به صورت پایین به بالا پر می‌شود. حالت $dp[i][j]$ نشان‌دهنده تعداد حالت‌های رسیدن به مجموع j با استفاده از i تاس است:

$$dp[i][j]=\sum_{\substack{k=1\\ j-k\geq 0}}^{m} dp[i-1][j-k]$$

با مقادیر اولیه $dp[0][0]=1$ و $dp[0][j]=0$ (برای $j>0$).
مثال: فرض کنید دو تاس شش‌وجهی داریم ($n=2,m=6,x=5$). برای رسیدن به مجموع ۵ داریم:

$$dp[2][5]=dp[1][4]+dp[1][3]+dp[1][2]+dp[1][1]=1+1+1+1=4$$

حالت‌ها عبارت‌اند از: $(1,4),(2,3),(3,2),(4,1)$؛ پس تعداد حالت‌های ممکن برابر ۴ است.
تحلیل پیچیدگی الگوریتم:

- **پیچیدگی زمانی:** $\mathcal{O}(n\cdot x\cdot m)$
- **پیچیدگی حافظه:** $\mathcal{O}(n\cdot x)$
- **بهینه‌سازی حافظه:** چون هر لایه $dp[i]$ فقط به لایهٔ $dp[i-1]$ وابسته است، با نگه داشتن دو سطر حافظه به $\mathcal{O}(x)$ کاهش می‌یابد.

سؤال دوم - خرد کردن سکه

فرض کنید n نوع سکه داریم که مقدار هر سکه مشخص است و از هر نوع سکه می‌توان به تعداد نامحدود استفاده کرد. الگوریتمی مبتنی بر برنامه‌نویسی پویا طراحی کنید که برای یک مبلغ مشخص x، کمترین تعداد سکه مورد نیاز برای تشکیل این مبلغ را محاسبه کند. در پاسخ خود:

۱. رابطه بازگشتی الگوریتم را بنویسید.

۲. پیچیدگی زمانی و حافظه‌ای الگوریتم را تحلیل کنید.

۳. مثال: سکه‌ها برابر با $\{1, 3, 4\}$ و مبلغ هدف $X = 6$ است. مراحل اجرای الگوریتم و نحوه رسیدن به جواب را برای این مثال توضیح دهید.

پاسخ تشریحی

۱. تعریف حالت و رابطه بازگشتی: فرض می‌کنیم $dp[i]$ برابر با کمترین تعداد سکه مورد نیاز برای تشکیل مبلغ i باشد. بنابراین جواب نهایی مسئله در $dp[x]$ قرار می‌گیرد.

- **حالت پایه (Base Cases):** برای مبلغ صفر به هیچ سکه‌ای نیاز نداریم ($dp[0] = 0$). برای سایر مقادیر ($i > 0$)، مقدار اولیه را بی‌نهایت در نظر می‌گیریم ($dp[i] = \infty$).

برای هر مبلغ i از 1 تا x و برای هر سکه با مقدار $c \in \text{coins}$ که $c \leq i$ باشد، رابطه بازگشتی به صورت زیر است:

$$dp[i] = \min_{c \in \text{coins}, c \leq i} (dp[i - c] + 1)$$

زیرا با اضافه کردن یک سکه با ارزش c به بهترین جواب مبلغ $i - c$، مبلغ i ساخته می‌شود.

۲. تحلیل پیچیدگی:

- **پیچیدگی زمانی:** برای محاسبه هر یک از x خانه جدول، باید روی تمام n نوع سکه پیمایش کنیم؛ بنابراین پیچیدگی زمانی برابر $\mathcal{O}(n \cdot x)$ است.
- **پیچیدگی حافظه:** تنها به یک آرایه یک‌بعدی به طول $x+1$ نیاز داریم؛ بنابراین پیچیدگی حافظه برابر $\mathcal{O}(x)$ است.

۳. بررسی مثال ($\text{coins} = \{1, 3, 4\}$ و $X = 6$): مقادیر جدول از 0 تا 5 به صورت زیر محاسبه می‌شوند:

- $dp[0] = 0$
- $dp[1] = dp[0] + 1 = 1$
- $dp[2] = dp[1] + 1 = 2$
- $dp[3] = \min(dp[2] + 1,\ dp[0] + 1) = \min(3, 1) = 1$
- $dp[4] = \min(dp[3] + 1,\ dp[1] + 1,\ dp[0] + 1) = \min(2, 2, 1) = 1$
- $dp[5] = \min(dp[4] + 1,\ dp[2] + 1,\ dp[1] + 1) = \min(2, 3, 2) = 2$

برای مبلغ هدف 6 داریم:

$$dp[6] = \min(dp[5] + 1,\ dp[3] + 1,\ dp[2] + 1) = \min(3, 2, 3) = 2$$

بنابراین با انتخاب دو سکه ۳ واحدی ($6 = 3 + 3$)، کمترین تعداد سکه مورد نیاز برابر ۲ است.

نکته (مبلغ غیرقابل‌ساخت): اگر پس از پر کردن جدول $dp[x] = \infty$ بماند، یعنی مبلغ x با هیچ ترکیبی از سکه‌ها ساخته نمی‌شود و خروجی الگوریتم باید -1 باشد (در مثال بالا چون سکهٔ ۱ داریم، همهٔ مبالغ ساخته می‌شوند).

سؤال سوم - طولانی‌ترین زیردنبالهٔ مشترک

دو رشتهٔ زیر را در نظر بگیرید:

$$A = \text{COMPUTER}, \quad B = \text{COMPILER}$$

می‌خواهیم با استفاده از برنامه‌ریزی پویا، طول طولانی‌ترین زیردنبالهٔ مشترک (LCS) این دو رشته را پیدا کنیم.

(الف) $dp[i][j]$ را به صورت ساده توضیح دهید که نشان می‌دهد چه چیزی را نگه می‌دارد.

(ب) اگر کاراکترهای مورد نظر در دو رشته برابر باشند، چه رابطه‌ای برای $dp[i][j]$ داریم؟

(ج) اگر این دو کاراکتر برابر نباشند، $dp[i][j]$ چگونه محاسبه می‌شود؟

(د) با استفاده از جدول DP طول طولانی‌ترین زیردنبالهٔ مشترک دو رشته را به دست آورید.

(هـ) یکی از طولانی‌ترین زیردنباله‌های مشترک را بنویسید.

پاسخ تشریحی

(الف) تعریف $dp[i][j]$: مقدار $dp[i][j]$ نشان‌دهنده طول طولانی‌ترین زیردنبالهٔ مشترک بین پیشوند به طول i از رشته اول ($A[1..i]$) و پیشوند به طول j از رشته دوم ($B[1..j]$) است. برای حالت‌های پایه که یکی از رشته‌ها تهی است، $dp[i][0] = 0$ و $dp[0][j] = 0$ می‌باشد.

(ب) در صورت برابری کاراکترها ($A[i] = B[j]$): اگر کاراکتر i-ام رشته A با کاراکتر j-ام رشته B برابر باشد:

$$dp[i][j] = dp[i-1][j-1] + 1$$

(ج) در صورت نابرابری کاراکترها ($A[i] \neq B[j]$): اگر دو کاراکتر برابر نباشند:

$$dp[i][j] = \max(dp[i-1][j],\ dp[i][j-1])$$

(د) جدول DP: جدول مقادیر $dp[i][j]$ برای دو رشته $A = \text{COMPUTER}$ و $B = \text{COMPILER}$ به صورت زیر است:

A \B	∅	C	O	M	P	I	L	E	R
∅	0	0	0	0	0	0	0	0	0
C	0	1	1	1	1	1	1	1	1
O	0	1	2	2	2	2	2	2	2
M	0	1	2	3	3	3	3	3	3
P	0	1	2	3	4	4	4	4	4
U	0	1	2	3	4	4	4	4	4
T	0	1	2	3	4	4	4	4	4
E	0	1	2	3	4	4	4	5	5
R	0	1	2	3	4	4	4	5	**6**

بر اساس خانهٔ آخر جدول ($dp[8][8]$)، طول طولانی‌ترین زیردنبالهٔ مشترک برابر **۶** است.
(هـ) زیردنبالهٔ مشترک: با بازگشت به عقب (Backtracking) از خانه $dp[8][8]$ در جدول، طولانی‌ترین زیردنبالهٔ مشترک یکتا به صورت **COMPER** به دست می‌آید.

سؤال چهارم - شکستن چوب

یک چوب به طول ۱۰ داریم که باید در سه نقطهٔ ۲، ۴ و ۷ بریده شود. در هر مرحله، هزینهٔ بریدن چوب برابر با طول قطعه‌ای است که در آن برش انجام می‌شود. ترتیب انجام برش‌ها آزاد است. مثلاً اگر ابتدا در نقطهٔ ۴ برش بزنیم، هزینهٔ این برش برابر ۱۰ خواهد بود.

(الف) اگر $dp[l][r]$ کمترین هزینهٔ لازم برای انجام تمام برش‌های بین دو سر بازهٔ l و r باشد، رابطهٔ بازگشتی $dp[l][r]$ را بنویسید.

(ب) کمترین هزینهٔ ممکن برای انجام هر سه برش را به دست آورید.

پاسخ تشریحی

(الف) رابطهٔ بازگشتی: نقاط برش را به همراه دو سر چوب در آرایهٔ مرتب $P = (p_0, p_1, p_2, p_3, p_4) = (0, 2, 4, 7, 10)$ قرار می‌دهیم. در ادامه l و r **اندیس‌های این آرایه** هستند (نه خود طول‌ها) و $dp[l][r]$ کمترین هزینهٔ بریدن تمام نقاط برش واقع بین p_l و p_r است:

- **حالت پایه:** اگر هیچ نقطهٔ برشی اکیداً بین p_l و p_r وجود نداشته باشد (یعنی $r = l + 1$)، $dp[l][r] = 0$.
- **گام بازگشتی:** برای $r \geq l + 2$، اولین برشِ این قطعه را در نقطهٔ p_k می‌زنیم. هزینهٔ این برش برابر طول قطعه، یعنی $p_r - p_l$ است و دو قطعهٔ حاصل مستقل از هم برش می‌خورند:

$$dp[l][r] = (p_r - p_l) + \min_{l<k<r} \big(dp[l][k] + dp[k][r]\big)$$

پاسخ نهایی مقدار $dp[0][4]$ است.
(ب) محاسبهٔ کمترین هزینه: محاسبه را بر اساس تعداد نقاط برش درونی از کوچک به بزرگ انجام می‌دهیم:

- **بازه‌های بدون نقطه برش میانی:** $dp[0][1] = dp[1][2] = dp[2][3] = dp[3][4] = 0$.
- **بازه‌های شامل ۱ نقطه برش میانی:**

$$dp[0][2] = (p_2 - p_0) + dp[0][1] + dp[1][2] = 4 - 0 = 4 \quad (\text{بازهٔ } [0, 4] \text{ برش در } 2)$$
$$dp[1][3] = (p_3 - p_1) + dp[1][2] + dp[2][3] = 7 - 2 = 5 \quad (\text{بازهٔ } [2, 7] \text{ برش در } 4)$$
$$dp[2][4] = (p_4 - p_2) + dp[2][3] + dp[3][4] = 10 - 4 = 6 \quad (\text{بازهٔ } [4, 10] \text{ برش در } 7)$$

- **بازه‌های شامل ۲ نقطه برش میانی:**

$$dp[0][3] = (p_3 - p_0) + \min\big(dp[0][1] + dp[1][3],\ dp[0][2] + dp[2][3]\big) = 7 + \min(5, 4) = 11$$
$$dp[1][4] = (p_4 - p_1) + \min\big(dp[1][2] + dp[2][4],\ dp[1][3] + dp[3][4]\big) = 8 + \min(6, 5) = 13$$

- **کل چوب ($dp[0][4]$):**

$$\begin{aligned} dp[0][4] &= (p_4 - p_0) + \min\big(dp[0][1] + dp[1][4],\ dp[0][2] + dp[2][4],\ dp[0][3] + dp[3][4]\big) \\ &= 10 + \min(13, 10, 11) = 10 + 10 = \mathbf{20} \end{aligned}$$

بنابراین کمترین هزینهٔ ممکن برابر **۲۰** است. ترتیب بهینه: ابتدا برش در ۴ (هزینهٔ ۱۰)، سپس برش در ۲ روی قطعهٔ $[0, 4]$ (هزینهٔ ۴) و برش در ۷ روی قطعهٔ $[4, 10]$ (هزینهٔ ۶).

سؤال پنجم - کاوشگر فضایی

یک کاوشگر فضایی قرار است برای یک مأموریت کوتاه به یک سیارک نزدیک فرستاده شود. تیم مأموریت باید از میان n ابزار علمی نامزد، زیرمجموعه‌ای را برای نصب روی کاوشگر انتخاب کند. کاوشگر دو محدودیت هم‌زمان دارد: حداکثر جرم قابل حمل M کیلوگرم و حداکثر توان الکتریکی پیوسته‌ای که می‌تواند تأمین کند E وات. ابزار شماره i دارای جرم m_i، مصرف توان e_i و امتیاز ارزش علمی v_i است. هر ابزار یا به طور کامل نصب می‌شود یا اصلاً نصب نمی‌شود. هدف ما انتخاب زیرمجموعه‌ای است که مجموع جرم آن از M بیشتر نشود، مجموع مصرف توان آن از E بیشتر نشود و در عین حال مجموع ارزش علمی به حداکثر برسد.

(الف) این مسئله را با رویکرد برنامه‌ریزی پویا مدل‌سازی کنید. یک تابع بازگشتی $e[i][a][b]$ تعریف کنید (بیشترین ارزش علمی قابل دستیابی با i ابزار اول با بودجه جرمی a و بودجه توانی b). رابطه بازگشتی و حالت‌های پایه را بنویسید.

(ب) پیچیدگی زمانی و مکانی این الگوریتم را بر حسب n، M و E تحلیل کنید و توضیح دهید چرا این پیچیدگی «شبه‌چندجمله‌ای» (pseudo-polynomial) نامیده می‌شود.

(ج) مثال عددی زیر را در نظر بگیرید ($M = 10$ کیلوگرم، $E = 8$ وات):

- ابزار A: جرم ۶، توان ۴، ارزش ۸
- ابزار B: جرم ۵، توان ۳، ارزش ۶
- ابزار C: جرم ۵، توان ۴، ارزش ۵

جدول برنامه‌ریزی پویای خود را ترسیم/دنبال کنید و نشان دهید چگونه به پاسخ نهایی می‌رسید، کدام ابزارها انتخاب می‌شوند و مقدار نهایی چقدر است.

پاسخ تشریحی

تعریف زیرمسئله: $e[i][a][b]$ بیشترین ارزش علمی است که با انتخاب از میان i ابزار اول به دست می‌آید، به شرطی که مجموع جرم حداکثر a و مجموع توان حداکثر b باشد.
حالت پایه: وقتی هیچ ابزاری در اختیار نداریم، ارزش صفر است.

$$e[0][a][b] = 0 \quad \text{all for} 0 \le a \le M,\ 0 \le b \le E$$

رابطه بازگشتی: برای ابزار i دو حالت داریم: آن را برنداریم، یا اگر جا شود برداریم.

$$e[i][a][b] = \begin{cases} e[i-1][a][b] & \text{if} m_i > a \text{ or } p_i > b \\ \max\left(e[i-1][a][b],\ v_i + e[i-1][a-m_i][b-p_i]\right) & \text{otherwise} \end{cases}$$

پاسخ نهایی: $e[n][M][E]$
در جواب بهینه، ابزار i یا حضور ندارد، پس جواب همان جواب بهینه برای $i-1$ ابزار با همان بودجه است یا حضور دارد، بقیه جواب باید بهینه‌ترین انتخاب از $i-1$ ابزار با بودجه باقی‌مانده باشد.

پیچیدگی زمانی: زمان کل، تعداد خانه‌های جدول ضربدر هزینه هر خانه است. اندیس i از 0 تا n، اندیس a از 0 تا M و اندیس b از 0 تا E تغییر می‌کند، پس جدول $(n+1)(M+1)(E+1)$ خانه

دارد. هر خانه فقط با دو مقایسه، خواندن دو خانه قبلی، یک جمع و یک $\max$ پر می‌شود، پس $O(1)$ است.

$$(n+1)(M+1)(E+1) = nME + nM + nE + ME + n + M + E + 1$$

چون برای $n, M, E \geq 1$ هر کدام از این ۸ جمله حداکثر nME است، پس کل عبارت حداکثر $8 \cdot nME$ است. چون الگوریتم همه خانه‌ها را پر می‌کند، کران پایین هم nME است، پس:

$$T = \Theta(n \cdot M \cdot E)$$

پیچیدگی مکانی: $\Theta(n \cdot M \cdot E)$ در نسخه‌ای که کل جدول نگه داشته می‌شود، پس مکان $\Theta(M \cdot E)$ یک نسخه بهینه‌تر است (چون لایه i فقط به لایه $i-1$ وابسته است، لایه‌های قدیمی‌تر لازم نیستند و می‌توان فقط با یک جدول دوبعدی کار کرد، a و b را از بزرگ به کوچک پیمایش کنیم چون خانه $dp[a - m_i][b - p_i]$ اندیس‌های کوچک‌تری دارد. با پیمایش نزولی، وقتی به $dp[a][b]$ می‌رسیم آن خانه هنوز به‌روز نشده و مقدار لایه قبلی $(i-1)$ را دارد.)
چرا شبه‌چندجمله‌ای؟ پیچیدگی هر الگوریتم بر حسب طول ورودی سنجیده می‌شود، یعنی تعداد بیت‌هایی که برای نوشتن ورودی لازم است. عدد M فقط با حدود $\log_2 M$ بیت نوشته می‌شود، طول ورودی لگاریتمی زیاد می‌شود، اما کار الگوریتم متناسب با خود مقدار M زیاد می‌شود.
اگر M با k بیت نوشته شود، مقدارش می‌تواند تا 2^k برسد. پس

$$n \cdot M \cdot E = n \cdot 2^{\log M} \cdot 2^{\log E}$$

یعنی زمان نسبت به تعداد بیت‌های M و E نمایی است. هر بیتی که به M اضافه شود، زمان اجرا را دو برابر می‌کند، در حالی که ورودی فقط یک بیت بزرگ‌تر شده است. عبارت nME ظاهر یک چندجمله‌ای را دارد، اما چندجمله‌ای بر حسب مقدار عددها است، نه بر حسب طول ورودی. در چندجمله‌ای واقعی زمان فقط به تعداد و طول داده‌ها بستگی دارد اما در شبه‌چندجمله‌ای زمان به بزرگی عددها هم بستگی دارد.

(ج) جدول سه‌بعدی است، پس برای هر i یک لایه دوبعدی رسم می‌کنیم؛ سطرها a (بودجه جرم) و ستون‌ها b (بودجه توان) هستند.
لایه $i = 0$: همه خانه‌ها صفر
لایه $i = 1$ (فقط A در اختیار): اگر $a \geq 6$ و $b \geq 4$ باشد مقدار 8، وگرنه 0.

A/b	0	1	2	3	4	5	6	7	8
0 – 5	0	0	0	0	0	0	0	0	0
6 – 10	0	0	0	0	8	8	8	8	8

لایه $i = 2$ (A و B): با $e[2][a][b] = \max\big(e[1][a][b],\ 6 + e[1][a-5][b-3]\big)$. چون $a - 5 \leq 5$ است، $e[1][a-5][b-3] = 0$ همیشه و گزینه برداشت B ارزش 6 دارد. (A و B با هم جرم 11 دارند و جا نمی‌شوند.)

A/b	0	1	2	3	4	5	6	7	8
0 – 4	0	0	0	0	0	0	0	0	0
5	0	0	0	6	6	6	6	6	6
6 – 10	0	0	0	6	8	8	8	8	8

لایه $i = 3$ (A و B و C): با $e[3][a][b] = \max\left(e[2][a][b],\ 5 + e[2][a-5][b-4]\right)$ گزینه برداشت C فقط وقتی سودمند است که $a - 5 = 5$ و $b - 4 \geq 3$. $a = 10$ و $b \geq 7$.

A/b	0	1	2	3	4	5	6	7	8
0 – 4	0	0	0	0	0	0	0	0	0
5	0	0	0	6	6	6	6	6	6
6 – 9	0	0	0	6	8	8	8	8	8
10	0	0	0	6	8	8	8	11	11

$$e[3][10][8] = \max\left(\underbrace{e[2][10][8]}_{8},\ 5 + \underbrace{e[2][5][4]}_{6}\right) = \max(8, 11) = 11$$

$e[3][10][8] = 11 \neq e[2][10][8] = 8$ پس C انتخاب شده؛ به $e[2][10-5][8-4] = e[2][5][4]$ می‌رویم.

$e[2][5][4] = 6 \neq e[1][5][4] = 0$ پس B انتخاب شده. به $e[1][5-5][4-3] = e[1][0][1]$ می‌رویم.

$e[1][0][1] = 0 = e[0][0][1]$ پس A انتخاب نشده.

پس با انتخاب C، B مجموع جرم 10 و مجموع توان 7 و بیشترین ارزش علمی 11 در حالی که با گریدی ابتدا A را برمی‌دارد بعد از آن با جرم باقی‌مانده 4 هیچ ابزار دیگری جا نمی‌شود و جواب 8 می‌شود.

سؤال ششم - ردیابی گله مهاجر

دو زیست‌شناس هر کدام یک فصل را صرف ردیابی یک گلهٔ مهاجر کرده‌اند و به طور مستقل به ترتیب زمانی، شمارهٔ هر آبشخوری که گله در آن فصل از آن بازدید کرده را یادداشت کرده‌اند. یک آبشخور می‌تواند تکرار شود و طول دو دفترچه می‌تواند متفاوت باشد. آن‌ها در آزمایشگاه می‌خواهند طولانی‌ترین مسیر اصلی ممکن را بازسازی کنند؛ دنباله‌ای از بازدیدها که با همان ترتیب نسبی در هر دو دفترچه دیده می‌شود. فرض کنید دو دفترچه $X = x_1 x_2 \dots x_n$ و $Y = y_1 y_2 \dots y_m$ باشند.

(الف) رابطه بازگشتی مناسب را تعریف کنید ($c[i][j]$ طول طولانی‌ترین مسیر اصلی با استفاده از i عنصر اول X و j عنصر اول Y) و حالت‌های پایه را مشخص کنید.

(ب) ثابت کنید اگر $x_n = y_m$، این عنصر مشترک حتماً باید در یکی از طولانی‌ترین مسیرهای اصلی حضور داشته باشد و باقی‌ماندهٔ آن مسیر بهینه، خود یک مسیر اصلی بهینه برای $n-1$ عنصر اول X و $m-1$ عنصر اول Y است (از استدلال برش و چسب استفاده کنید).

(ج) توضیح دهید چرا نسخهٔ بازگشتی ساده (بدون ذخیرهٔ نتایج) این الگوریتم برای دو دفترچه با طول ۴۰ عملاً هرگز تمام نمی‌شود و پیچیدگی زمانی و مکانی نسخهٔ درست DP را بر حسب n و m بیان کنید.

پاسخ تشریحی

تعریف: $c[i][j]$ طول طولانی‌ترین زیردنباله مشترک

$$X_i = x_1 \dots x_i, \qquad Y_j = y_1 \dots y_j$$

اگر یکی از دفترچه‌ها خالی باشد، مسیر مشترکی وجود ندارد، اگر آخرین آبشخورها یکسان باشند، آن را جزو مسیر می‌گیریم و مسئله را روی بقیه حل می‌کنیم، اگر یکسان نباشند، دست‌کم یکی از آن دو در جواب نقشی ندارد. پس یکی را حذف می‌کنیم و بهترین حالت را برمی‌داریم.

$$c[i][j] = \begin{cases} 0 & \text{if} i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & \text{if} x_i = y_j \\ \max\left(c[i-1][j],\, c[i][j-1]\right) & \text{if} x_i \neq y_j \end{cases}$$

پاسخ نهایی: $c[n][m]$

(ب) فرض کنید $x_n = y_m$ و $Z = z_1 z_2 \dots z_k$ یک LCS دلخواه از X و Y باشد.

۱) عنصر مشترک حتماً در یک LCS حضور دارد: $z_k = x_n = y_m$

فرض خلف: $z_k \neq x_n$ z_k نمی‌تواند به موقعیت n در X یا موقعیت m در Y نگاشته شده باشد. پس تمام Z درون Y_{m-1} و X_{n-1} است و حال x_n را به انتهای Z اضافه می‌کنیم: $Z' = Z \cdot x_n$ x_n و y_m هر دو بعد از همه عناصر Z آمده‌اند. پس این دنباله یک زیردنباله مشترک از X و Y است ولی طولش $k+1$ است که مسلماً بیشتر از k است و با بهینه بودن Z در تناقض است. پس $z_k = x_n = y_m$: اگر z_k به رخداد قبلی همین نماد نگاشته شده باشد، می‌توانیم نگاشت آن را به موقعیت n و m منتقل کنیم.

۲) $Z_{k-1} = z_1 \dots z_{k-1}$ یک LCS از X_{n-1} و Y_{m-1} است.

مشترک بودن: چون z_k با x_n و y_m جفت شده، بقیه عناصر Z درون X_{n-1} و Y_{m-1} قرار دارند. پس

Z_{k-1} زیردنباله مشترک آن‌هاست.
بهینه بودن (برش و چسب): فرض کنید یک زیردنباله مشترک از X_{n-1} و Y_{m-1} با طول بیشتر از $k-1$ وجود داشته باشد. Z_{k-1} را از جواب برش می‌زنیم و آن را به جایش می‌چسبانیم اگر آن را W بنامیم: $W \cdot x_n$ یک زیردنباله مشترک از X و Y است با طول $|W|+1 > k$. این با بهینه بودن Z تناقض دارد.
هر جواب بهینه از یک جواب بهینه برای زیرمسئله کوچک‌تر به‌علاوه عنصر مشترک تشکیل می‌شود.

(ج) چرا نسخه بازگشتی ساده تمام نمی‌شود؟ زیرمسئله‌ها بارها تکرار می‌شوند.

$$\begin{array}{cccc} & & (n,m) & \\ & (n-1,m) & & (n,m-1) \\ (n-2,m) & & (n-1,m-1) & & (n-1,m-1) & & (n,m-2) \\ \cdots & & & & \cdots \end{array}$$

هر فراخوانی $i+j$ را یک واحد کم می‌کند و بازگشت فقط وقتی متوقف می‌شود که $i=0$ یا $j=0$ شود. پس هیچ شاخه‌ای قبل از عمق $\min(n,m)$ تمام نمی‌شود و درخت تا این عمق یک درخت دودویی کامل است:

$$2^{\min(n,m)} = 2^{40} \approx 10^{12}$$

در بدترین حالت (مثلاً وقتی هیچ نماد مشترکی وجود ندارد) هر فراخوانی غیرپایه دو فراخوانی دیگر ایجاد می‌کند:

$$T(0,m) = T(n,0) = 1, \qquad T(n,m) = T(n-1,m) + T(n,m-1) + 1$$

دقیقاً $2\binom{n+m}{n} - 1$ فراخوانی می‌دهد. برای $n = m = 40$:

$$2\binom{80}{40} \approx 2 \times 10^{23}$$

با سرعت 10^9 فراخوانی در ثانیه، این کار حدود نزدیک ۷ میلیون سال طول می‌کشد. کل زیرمسئله‌های متمایز فقط

$$(n+1)(m+1) = 41 \times 41 = 1681$$

تا هستند. الگوریتم ساده به جای اینکه هر کدام را یک بار حل کند، آن‌ها را میلیاردها میلیارد بار از نو حل می‌کند.
زمانی: $\Theta(n \cdot m)$، چون $(n+1)(m+1)$ خانه داریم و هر خانه در $O(1)$ پر می‌شود.
مکانی: $\Theta(n \cdot m)$ برای نگه داشتن کل جدول، که برای بازسازی مسیر لازم است. اگر فقط طول مسیر را بخواهیم، چون هر سطر فقط به سطر قبلی وابسته است، با دو سطر می‌توان مکان را به $\Theta(\min(n,m))$ رساند.